МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА — Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

Практические задания
«Выбор темы проекта. Сбор требований»
«Написание документации проекта. UML-диаграммы»
по дисциплине «Технологии индустриального программирования»

Тема проекта:

«Визуализатор алгоритмов сортировки»

Software Requirements Specification (SRS)

Выполнили: студенты группы ЭФБО-04-25
Терешин Максим Геннадьевич
Галкин Антон Дмитриевич

Преподаватель: Афонин Сергей Александрович

Москва 2026

Содержание

1. Введение **3**

1.1. Цель 3

1.2. Область действия (Scope) 3

1.3. Определения и аббревиатуры 4

2. Общее описание **5**

2.1. Описание продукта 5

2.2. Функции продукта 5

2.3. Характеристики пользователей 5

3. Детальные требования **6**

3.1. Методология сбора требований 6

3.2. Бизнес-требования 7

3.3. Пользовательские требования 7

3.4. Функциональные требования 8

3.5. Нефункциональные требования 9

3.6. Матрица прослеживаемости 10

4. UML-диаграммы **11**

4.1. Диаграмма вариантов использования (Use Case Diagram) 11

4.2. Диаграмма классов (Class Diagram) 13

4.3. Диаграмма последовательности (Sequence Diagram) 15

1. Введение

1.1. Цель

Настоящий документ представляет собой спецификацию требований к программному обеспечению (SRS) проекта «Визуализатор алгоритмов сортировки». Документ фиксирует результаты этапа инициации проекта: выбранную тему и границы, методологию сбора требований, иерархию требований по Вигерсу (бизнес-, пользовательские, функциональные и нефункциональные требования) и UML-модели будущей системы.

Документ предназначен для разработчика проекта и преподавателя дисциплины и служит основой для последующих этапов проектирования, реализации и тестирования.

1.2. Область действия (Scope)

Проект — настольное приложение на языке C++, которое наглядно показывает пошаговую работу классических алгоритмов сортировки (пузырьком, быстрой и слиянием) на анимированной столбчатой диаграмме. Приложение выступает учебным пособием для студентов, изучающих курс «Алгоритмы и структуры данных», и демонстрационным инструментом для преподавателя.

В область действия входит:

1. реализация трёх алгоритмов сортировки (пузырьком, быстрая, слиянием) собственными силами, без использования готовых функций сортировки;
2. формирование исходных данных: генерация массива заданного размера и типа или ручной ввод;
3. визуализация процесса в реальном времени (пошаговое отображение сравнений и перестановок);
4. управление процессом: выбор алгоритма, запуск, пауза, возобновление, шаг вперёд и назад, сброс;
5. регулировка скорости анимации (от пошагового режима до быстрой прокрутки);
6. сбор и вывод статистики: количество сравнений, перестановок и затраченное время;
7. режим сравнения двух алгоритмов на одинаковых данных.

Не входит в область действия:

1. сетевое взаимодействие (клиент-серверная архитектура, удалённый доступ), учётные записи;
2. сохранение истории сортировок в базу данных или облачные сервисы;
3. сортировка данных, отличных от числовых массивов (строк, объектов, файлов, баз данных, данных реальных предприятий);
4. мобильные и веб-версии приложения;
5. использование готовых решений, подменяющих реализацию алгоритмов или базовую логику визуализации (допустимы библиотеки для окна, ввода и вывода графики).

Ограничения: язык реализации — C++; применение сторонних библиотек и сервисов допустимо, но не должно подменять собой реализацию самих алгоритмов сортировки и базовой логики визуализации.

1.3. Определения и аббревиатуры

Термин	Определение
SRS	Software Requirements Specification — спецификация требований к программному обеспечению
UML	Unified Modeling Language — унифицированный язык моделирования
Иерархия Вигерса	Разделение требований на бизнес-требования (БТ), пользовательские (ПТ), функциональные (ФТ) и нефункциональные (НФТ) по К. Вигерсу
User Story	Пользовательская история — короткое описание потребности по шаблону «Как <роль>, я хочу <действие>, чтобы <цель>»
MoSCoW	Метод приоритизации: M (Must) — обязательно, S (Should) — желательно, C (Could) — при наличии времени
Шаг (Step)	Элементарное действие алгоритма: сравнение двух элементов, обмен, запись значения
Журнал шагов (StepLog)	Упорядоченная последовательность шагов, записанная алгоритмом и воспроизводимая визуализатором
Сравнение / перестановка	Основные метрики вычислительной работы алгоритма сортировки, отображаемые в статистике
Временная сложность	Оценка роста времени работы алгоритма в нотации O-большое (лучший, средний и худший случаи)
FPS	Frames Per Second — число кадров в секунду

2. Общее описание

2.1. Описание продукта

«Визуализатор алгоритмов сортировки» — автономное учебное приложение для персонального компьютера (Windows и Linux), которое превращает абстрактный процесс сортировки чисел в наглядную анимацию. Программа отображает массив в виде столбцов, показывает сравнение и перестановку элементов, а также ведёт подсчёт затраченных ресурсов (времени и числа операций).

Принцип работы построен на разделении двух задач: алгоритм не рисует, а только записывает свои действия в журнал шагов, а отдельный блок воспроизведения читает журнал и показывает его на экране в нужном темпе. Благодаря этому пауза, шаг назад и смена скорости реализуются просто, а добавление нового алгоритма не требует изменений в отрисовке. Продукт является самостоятельным и не интегрируется с внешними системами.

2.2. Функции продукта

- выбор одного из трёх алгоритмов сортировки (пузырьком, быстрая, слиянием);
- создание исходного массива: случайный, по возрастанию, по убыванию, почти отсортированный, а также ввод вручную;
- анимированная визуализация сортировки с цветовой подсветкой сравниваемых, переставляемых и уже упорядоченных элементов;
- управление воспроизведением: старт, пауза, возобновление, шаг вперёд, шаг назад, сброс;
- регулировка скорости анимации;
- сбор и отображение статистики: количество сравнений, перестановок, время выполнения алгоритма;
- вывод краткого описания алгоритма и его сложности;
- режим сравнения двух алгоритмов бок о бок на одинаковых данных.

2.3. Характеристики пользователей

Группа	Описание	Уровень подготовки	Основные цели
Студент (основная аудитория)	Обучающийся по направлениям, связанным с программированием; изучает алгоритмы и структуры данных	Знает базовые понятия сортировки, не обязательно понимает ход конкретного алгоритма; от начального до продвинутого владения ПК	Понять, как алгоритм работает шаг за шагом; сравнить алгоритмы между собой
Преподаватель	Ведёт занятия по алгоритмам, демонстрирует материал на лекциях и практических занятиях	Хорошо знает алгоритмы; нужен надёжный и наглядный инструмент	Показать работу алгоритма на лекции; продемонстрировать разницу сложностей на одинаковых данных

Пользователям не требуются навыки администрирования; работа с приложением строится на мыши и клавиатуре через графический интерфейс на русском языке.

3. Детальные требования

3.1. Методология сбора требований

Проект учебный, поэтому заказчиком и ключевым пользователем выступает целевая аудитория — студенты курса «Алгоритмы и структуры данных» и преподаватель. Для сбора требований применён комбинированный подход, соответствующий размеру и характеру проекта.

1. **Анализ документов.** Изучены задание на практические занятия и ограничения проекта (язык C++, допустимость сторонних библиотек, запрет подмены реализации алгоритмов и визуализации готовыми решениями), а также материалы курса по алгоритмам сортировки (описания, оценки сложности). *Результат:* перечень алгоритмов и ограничения проекта.
2. **Интервью с условным заказчиком** — традиционный метод; помог определить бизнес-цели (обучение студентов) и ключевые пользовательские сценарии (потребность в наглядности и управлении скоростью). Заданные вопросы и полученные ответы приведены ниже.
3. **User Stories и Story Mapping** — agile-подход; потребности формализованы в виде пользовательских историй с критериями приёмки для двух ролей (студент, преподаватель). Истории выстроены по пути пользователя: выбрать алгоритм → подготовить данные → запустить → управлять → оценить результат. Приоритеты расставлены методом MoSCoW.
4. **Декомпозиция по Вигерсу.** От бизнес-целей к пользовательским историям, затем к функциям и ограничениям качества. Каждое требование получило идентификатор для прослеживаемости (раздел 3.6).

Результаты интервью:

Вопрос	**Ответ заказчика**
Какова главная цель приложения?	Визуальная демонстрация работы алгоритмов для обучения.
Какие алгоритмы обязательно нужно реализовать?	Базовые: пузырьком, быстрая, слиянием.
Как пользователь будет задавать данные?	Генерация случайного массива и ручной ввод.
Нужна ли статистика?	Да, важно видеть количество операций и время.
Как должно выглядеть управление?	Пауза, скорость, сброс — для детального разбора каждого шага.

Отделение потребностей от деталей реализации. В требования не включены решения, относящиеся к проектированию: выбор графической библиотеки и системы сборки, внутренние имена классов и форматы данных. Они фиксируются на этапе проектирования (раздел 4); в разделах 3.2–3.5 остались только потребности пользователей и измеримые свойства системы.

3.2. Бизнес-требования

Бизнес-требования отвечают на вопрос «зачем создаётся продукт» и описывают цели с точки зрения заказчика.

ID	Требование	Критерий достижения
БТ-1	Повысить наглядность изучения алгоритмов сортировки в курсе «Алгоритмы и структуры данных» и снизить порог понимания темы за счёт визуализации абстрактных операций	Студент по анимации может объяснить принцип каждого из реализованных алгоритмов
БТ-2	Дать преподавателю инструмент для демонстрации алгоритмов на занятии без подготовки презентаций	Запуск демонстрации выбранного алгоритма не более чем за 3 действия
БТ-3	Обеспечить количественное сравнение алгоритмов по числу сравнений, перестановок и времени работы на одинаковых данных	Для двух алгоритмов на одном массиве выводятся числовые метрики

3.3. Пользовательские требования

Пользовательские требования выражают, что пользователи хотят делать с продуктом. Оформлены в виде User Stories с критериями приёмки; приоритеты — по MoSCoW.

ID	User Story	Критерии приёмки	Приор.
ПТ-1	Как студент, я хочу выбрать алгоритм сортировки из списка, чтобы увидеть именно тот алгоритм, который изучаю	Доступны три алгоритма (пузырьком, быстрая, слиянием); выбранный алгоритм подсвечивается; смена алгоритма сбрасывает текущий прогресс	M
ПТ-2	Как студент, я хочу задать размер и тип исходного массива, чтобы увидеть работу алгоритма в лучшем, среднем и худшем случаях	Размер от 5 до 200 элементов; типы: случайный, по возрастанию, по убыванию, почти отсортированный; числа создаются в заданном диапазоне	M
ПТ-3	Как студент, я хочу ввести свой массив вручную, чтобы проверить алгоритм на примере из задачника	Числа вводятся через пробел или запятую; нечисловой ввод и число элементов вне диапазона 5–200 отклоняются с понятным сообщением	S
ПТ-4	Как студент, я хочу запускать, ставить на паузу, возобновлять и сбрасывать сортировку, чтобы изучать её в удобном темпе	«Пауза» останавливает анимацию без заметной задержки; «Продолжить» возобновляет с того же шага; «Сброс» возвращает массив в исходное состояние	M
ПТ-5	Как студент, я хочу переходить на шаг вперёд и назад, чтобы разобрать непонятный момент	Один шаг соответствует одному действию алгоритма; при шаге назад возвращаются состояние массива и счётчики	S
ПТ-6	Как студент, я хочу менять скорость анимации, чтобы замедлить сложный фрагмент или быстро дойти до конца	Регулятор скорости меняет задержку между шагами; изменение работает во время анимации без перезапуска	M

ID	User Story	Критерии приёмки	Приор.
ПТ-7	Как студент, я хочу видеть анимацию в виде столбцов и понимать, какие элементы сейчас сравниваются или меняются местами, чтобы не потерять ход алгоритма	Высота столбца пропорциональна значению; цвет различает обычные, сравниваемые, переставляемые и окончательно упорядоченные элементы; обновление не реже 30 кадров в секунду	M
ПТ-8	Как студент, я хочу видеть число сравнений, перестановок и время работы, чтобы оценить эффективность алгоритма	Счётчики увеличиваются в реальном времени; по завершении выводится итоговая статистика, время — в миллисекундах	M
ПТ-9	Как студент, я хочу прочитать краткое описание алгоритма и его сложность, чтобы связать картинку с теорией	Отображаются название, краткое описание и сложность (лучший, средний, худший случай)	S
ПТ-10	Как студент или преподаватель, я хочу запустить два алгоритма на одинаковых массивах и сравнить их метрики, чтобы наглядно увидеть разницу в скорости и числе операций	Оба алгоритма получают копии одного массива и показываются в двух панелях со своими счётчиками; по завершении выводится таблица сравнения (время и операции)	S
ПТ-11	Как преподаватель, я хочу, чтобы интерфейс можно было сделать крупным и контрастным, чтобы его было видно с задних рядов аудитории	Переключатель увеличивает шрифты и контраст без перезапуска приложения	C

3.4. Функциональные требования

Функциональные требования описывают, что система должна делать, и детализируют пользовательские требования.

ID	Требование	Источник	Приор.
ФТ-1	Система должна реализовывать алгоритмы сортировки пузырьком, быструю сортировку и сортировку слиянием собственным кодом, без использования готовых функций сортировки	ПТ-1	M
ФТ-2	Система должна фиксировать действия алгоритма (сравнение, обмен, запись значения) отдельно от их отображения, чтобы воспроизведение можно было приостанавливать, ускорять и отматывать	ПТ-4, ПТ-5, ПТ-7	M
ФТ-3	Система должна предоставлять список алгоритмов и позволять выбрать один из них до запуска и между запусками; смена алгоритма сбрасывает прогресс	ПТ-1	M
ФТ-4	Система должна генерировать массив размером от 5 до 200 элементов в режимах: случайный, по возрастанию, по убыванию, почти отсортированный	ПТ-2	M
ФТ-5	Система должна принимать массив, введённый вручную списком чисел через пробел или запятую, и проверять его корректность с выводом сообщения об ошибке	ПТ-3	S

ID	Требование	Источник	Приор.
ФТ-6	Система должна отображать массив столбчатой диаграммой, где высота столбца пропорциональна значению элемента, и обновлять её на каждом шаге	ПТ-7	M
ФТ-7	Система должна подсвечивать разными цветами элементы: обычные, сравниваемые, переставляемые, окончательно упорядоченные	ПТ-7	M
ФТ-8	Система должна обрабатывать команды «Старт», «Пауза», «Продолжить» и «Сброс»	ПТ-4	M
ФТ-9	Система должна обрабатывать команды «Шаг вперёд» и «Шаг назад»	ПТ-5	S
ФТ-10	Система должна позволять изменять скорость воспроизведения от 1 до 100 шагов в секунду, в том числе во время анимации	ПТ-6	M
ФТ-11	Система должна подсчитывать и отображать количество сравнений и перестановок в реальном времени; при шаге назад счётчики должны возвращаться к предыдущим значениям	ПТ-5, ПТ-8	M
ФТ-12	Система должна замерять время выполнения алгоритма (без учёта задержек анимации)	ПТ-8	M
ФТ-13	Система должна по завершении сортировки показать сообщение об окончании и итоговую статистику (сравнения, перестановки, время в мс)	ПТ-8	M
ФТ-14	Система должна показывать название выбранного алгоритма, его краткое описание и сложность (лучший, средний, худший случай)	ПТ-9	S
ФТ-15	Система должна позволять запустить два алгоритма на копиях одного массива, отобразить их в двух панелях со своими счётчиками и вывести таблицу сравнения по завершении	ПТ-10	S
ФТ-16	Система должна позволять включить крупный режим отображения (увеличенные шрифты и контраст)	ПТ-11	C

3.5. Нефункциональные требования

Нефункциональные требования описывают качества и ограничения, при которых система должна работать.

ID	Категория	Требование
НФТ-1	Ограничение реализации	Проект реализуется на языке C++ (стандарт C++17). Сторонние библиотеки допускаются, но не должны подменять реализацию алгоритмов и логики воспроизведения
НФТ-2	Переносимость	Приложение должно собираться и работать в Windows 10/11 и Linux (Ubuntu 22.04 и новее)
НФТ-3	Производительность	Плавность визуализации — не менее 30 кадров в секунду при массиве из 200 элементов на компьютере со встроенной графикой
НФТ-4	Отзывчивость	Реакция интерфейса на команды пользователя (пауза, сброс и т. д.) — не более 100 мс; подготовка всех шагов алгоритма для 200 элементов — не более 1 с
НФТ-5	Корректность	Результат каждого алгоритма должен совпадать с результатом эталонной сортировки; проверка выполняется автоматическими тестами не менее чем на 100 случайных массивах

ID	Категория	Требование
НФТ-6	Расширяемость	Добавление нового алгоритма должно требовать только реализации общего интерфейса алгоритма и регистрации в списке, без изменений в отрисовке и блоке воспроизведения
НФТ-7	Удобство использования	Новый пользователь должен научиться запускать сортировку не более чем за 5 минут без инструкции; интерфейс и подсказки — на русском языке
НФТ-8	Надёжность	Некорректный ввод (пустая строка, нечисловые значения, число элементов вне диапазона 5–200) не должен приводить к аварийному завершению; пользователю выводится сообщение об ошибке
НФТ-9	Сопровождаемость	Исходный код разделён на модули (алгоритмы, журнал шагов, воспроизведение, отрисовка, контроллер); имена и стиль оформления единообразны
НФТ-10	Ресурсы	Потребление оперативной памяти — не более 200 МБ; запуск не требует установки дополнительного ПО

3.6. Матрица прослеживаемости

Матрица показывает связь между уровнями требований: от бизнес-целей к пользовательским историям и далее к функциям. Нефункциональные требования (НФТ-1 — НФТ-10) распространяются на систему в целом.

Бизнес-требование	Пользовательские требования	Функциональные требования
БТ-1	ПТ-1 — ПТ-7, ПТ-9	ФТ-1 — ФТ-10, ФТ-14
БТ-2	ПТ-1, ПТ-4, ПТ-6, ПТ-10, ПТ-11	ФТ-3, ФТ-8, ФТ-10, ФТ-15, ФТ-16
БТ-3	ПТ-8, ПТ-10	ФТ-11 — ФТ-13, ФТ-15

4. UML-диаграммы

4.1. Диаграмма вариантов использования (Use Case Diagram)

Диаграмма отражает функциональные требования: акторы «Студент» и «Преподаватель» (преподаватель наследует все возможности студента и дополнительно может включить крупный режим) и прецеденты системы. Связь «include» показывает обязательное включение сценария, «extend» — необязательное расширение.

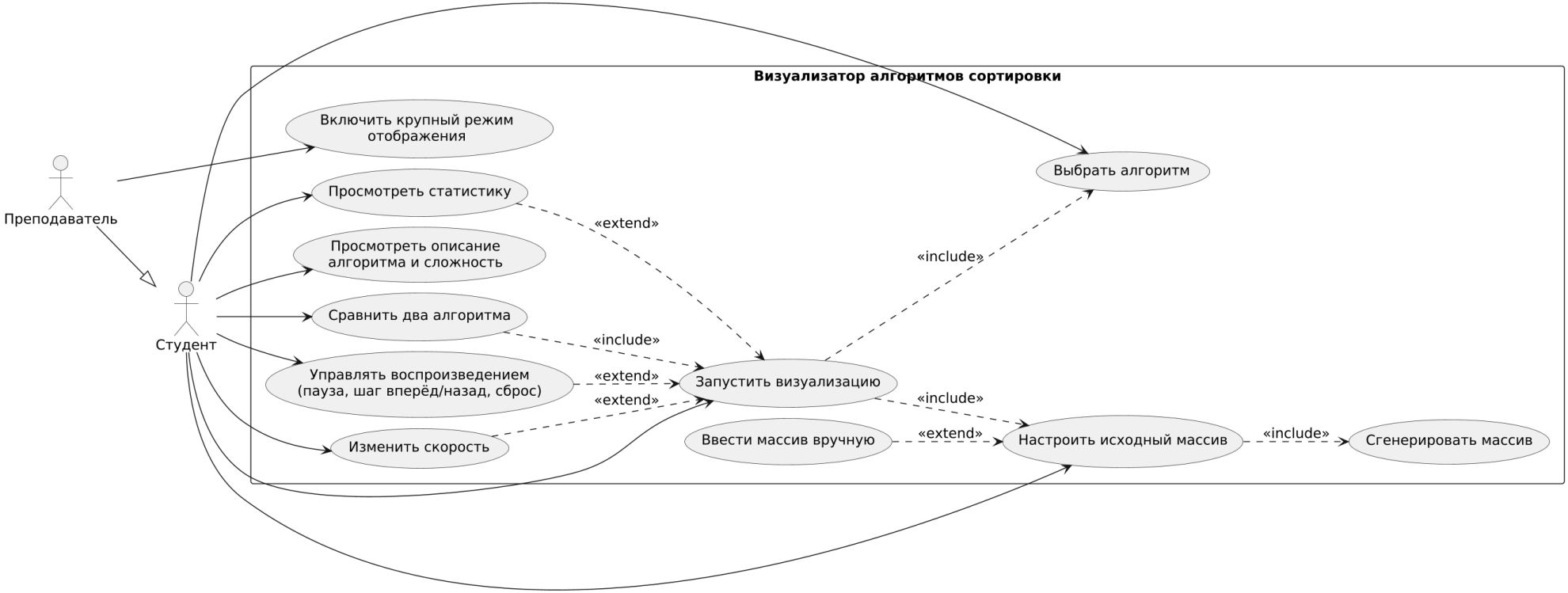


Рисунок 1 — Диаграмма вариантов использования

Пояснение к рисунку 1

Акторы: Студент; Преподаватель (обобщение — наследует все прецеденты студента).

Прецедент	Описание	Связи	Требования
Выбрать алгоритм	Выбор алгоритма из списка	включается в «Запустить визуализацию»	ПТ-1; ФТ-3
Настроить исходный массив	Задание размера и типа массива	включается в «Запустить визуализацию»; включает «Сгенерировать массив»	ПТ-2; ФТ-4
Сгенерировать массив	Создание массива в выбранном режиме	включается в «Настроить исходный массив»	ПТ-2; ФТ-4
Ввести массив вручную	Ввод чисел с проверкой корректности	расширяет «Настроить исходный массив»	ПТ-3; ФТ-5
Запустить визуализацию	Выполнение сортировки и анимация шагов	включает выбор алгоритма и настройку массива	ПТ-4, ПТ-7; ФТ-1, ФТ-2, ФТ-6, ФТ-7
Управлять воспроизведением	Пауза, возобновление, шаг вперёд и назад, сброс	расширяет «Запустить визуализацию»	ПТ-4, ПТ-5; ФТ-8, ФТ-9
Изменить скорость	Изменение скорости анимации	расширяет «Запустить визуализацию»	ПТ-6; ФТ-10
Просмотреть статистику	Счётчики сравнений и перестановок, время, итоговый отчёт	расширяет «Запустить визуализацию»	ПТ-8; ФТ-11 — ФТ-13
Просмотреть описание алгоритма и сложность	Название, описание и сложность выбранного алгоритма	—	ПТ-9; ФТ-14
Сравнить два алгоритма	Запуск двух алгоритмов на одинаковом массиве	включает «Запустить визуализацию»	ПТ-10; ФТ-15
Включить крупный режим отображения	Увеличение шрифтов и контраста (актор — Преподаватель)	—	ПТ-11; ФТ-16

4.2. Диаграмма классов (Class Diagram)

Диаграмма показывает статическую структуру системы: основные сущности, их атрибуты, методы и отношения (ассоциация, наследование, агрегация и композиция).

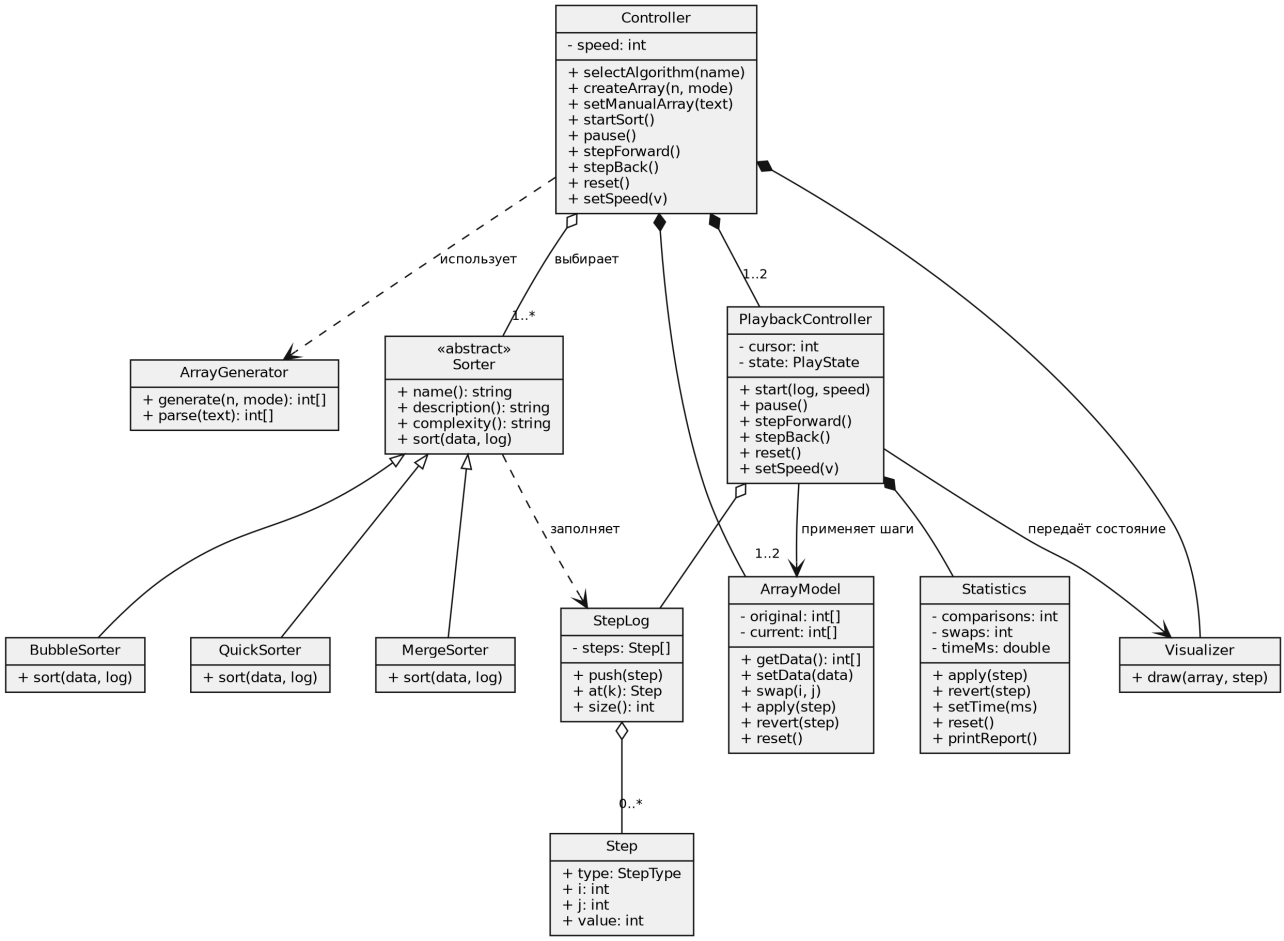


Рисунок 2 — Диаграмма классов

Кратности на диаграмме. Значение 1..2 у PlaybackController и ArrayModel означает: в обычном режиме создаётся по одному экземпляру, в режиме сравнения двух алгоритмов — по два (отдельная модель массива и блок воспроизведения для каждого алгоритма). Значение 1..* у Sorter — Controller хранит список из одного и более доступных алгоритмов. Значение 0..* у Step — журнал может содержать любое число шагов, включая ноль.

Пояснение к рисунку 2

Связи между классами. Алгоритмы наследуются от абстрактного класса Sorter и записывают шаги в StepLog (агрегация шагов Step). Controller композитно владеет моделью массива, блоками воспроизведения и отрисовкой и агрегирует доступные алгоритмы; PlaybackController агрегирует журнал шагов, композитно владеет статистикой и передаёт состояние в ArrayModel и Visualizer.

Назначение классов:

Класс	Назначение
Controller	Связывает компоненты, принимает команды пользователя (выбор алгоритма, создание массива, запуск, пауза, шаги, сброс, скорость); в режиме сравнения создаёт два блока воспроизведения
ArrayGenerator	Генерирует массив заданного размера и типа; разбирает и проверяет ручной ввод
ArrayModel	Хранит исходный и текущий массив, управляет доступом к элементам, применяет и откатывает шаги
Sorter (абстрактный)	Общий интерфейс алгоритма: название, описание, сложность, метод sort(); алгоритм ничего не рисует, а только заполняет журнал шагов
BubbleSorter, QuickSorter, MergeSorter	Реализации алгоритмов сортировки пузырьком, быстрой и слиянием (наследуются от Sorter)
Step, StepLog	Шаг (тип, индексы, значение) и упорядоченный журнал шагов
PlaybackController	Воспроизводит журнал: старт, пауза, шаг вперёд и назад, сброс, скорость; хранит положение курсора
Visualizer	Отрисовывает массив столбцами с подсветкой текущего шага
Statistics	Собирает метрики: сравнения, перестановки, время работы алгоритма; формирует итоговый отчёт

4.3. Диаграмма последовательности (Sequence Diagram)

Детализирован основной сценарий «Запуск визуализации сортировки»: Controller получает копию массива, вызывает выбранный алгоритм, который записывает журнал шагов; затем блок воспроизведения в цикле читает шаги, применяет их к массиву, обновляет статистику и перерисовывает экран; по завершении выводится итоговая статистика.

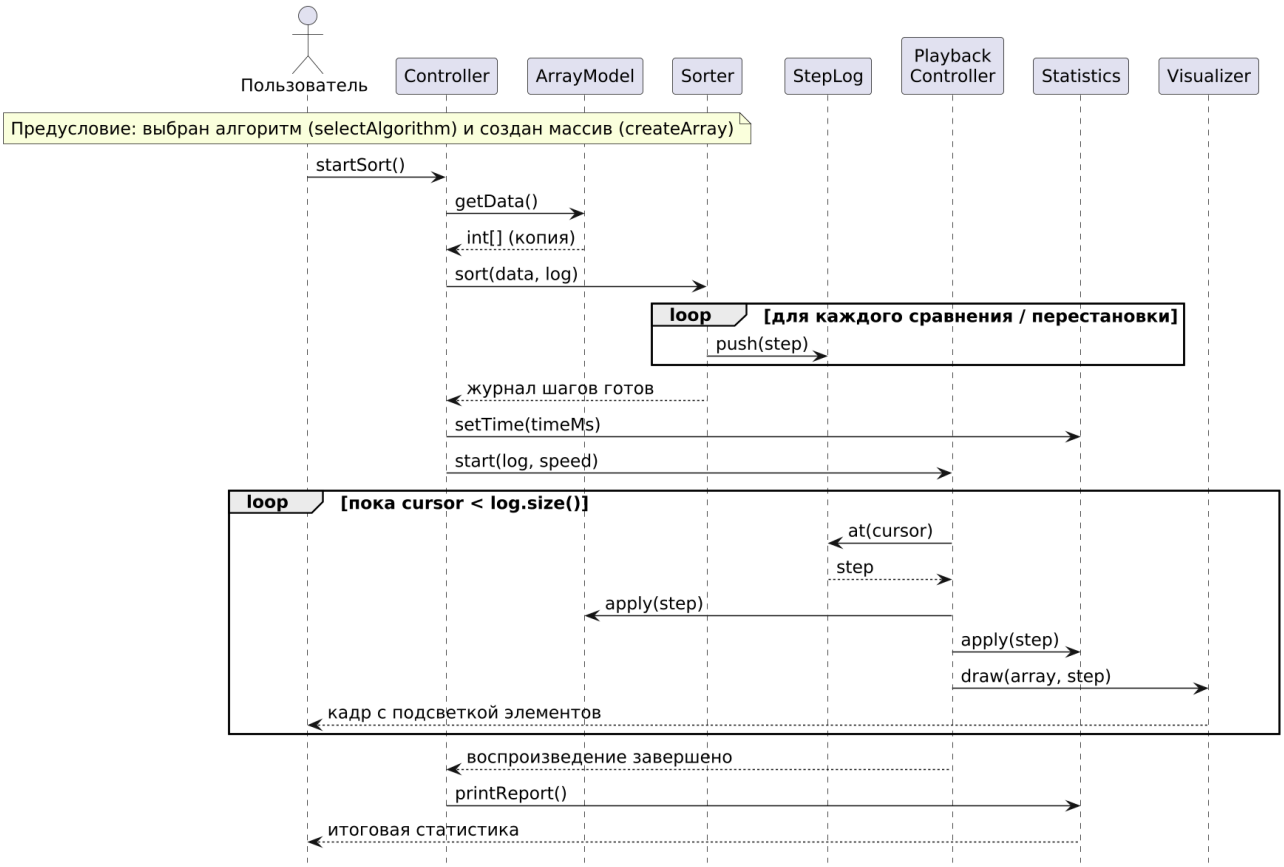


Рисунок 3 — Диаграмма последовательности сценария «Запуск визуализации сортировки»

Участники: Пользователь, Controller, ArrayModel, Sorter, StepLog, PlaybackController, Statistics, Visualizer.

Порядок сообщений:

1. Пользователь нажимает «Старт» (startSort у Controller); алгоритм и массив уже выбраны.
2. Controller запрашивает у ArrayModel копию исходного массива.
3. Controller передаёт массив и журнал шагов выбранному Sorter.
4. Sorter в цикле записывает каждое сравнение и перестановку в StepLog.
5. Controller передаёт в Statistics время работы алгоритма и запускает PlaybackController.
6. PlaybackController в цикле берёт очередной шаг из StepLog, применяет его к ArrayModel и Statistics.
7. PlaybackController просит Visualizer отрисовать текущий шаг; пользователь видит кадр с подсветкой элементов.
8. По завершении воспроизведения Controller запрашивает у Statistics итоговый отчёт и выводит его пользователю.